



I.E.S. "ÁGORA"

Dpto.: Departamento de Informática y Comunicaciones.

Ciclo Formativo de G. Superior: Desarrollo de
Aplicaciones Web.

FastFingers

Autor: Jorge Barriga Rubio

Tutora: María Carmen Moreno Cortés

Cáceres, a 11 de mayo de 2026

Índice

1. Propuesta de Proyecto	3
2. Introducción	5
a. Ficha de proyecto	
b. Objetivos	
c. Funcionalidades	
d. Justificación	
3. Diseño	11
a. Diseño de la base de datos	
b. Diseño del sitio web	
4. Aspectos interesantes del código	17
5. Manual de usuario	23
• Acceso a la Plataforma	
■ Inicio de sesión y registro	
• Modos de Juego	
■ Modo Contrarreloj Clásico y Modo Partida a Medida	
■ El Rosco	
• Modo Multijugador Online (Versus)	
■ Crear y unirse a una sala privada	
■ Pantalla de resultados cruzados (Victoria/Derrota)	
• Progresión y Economía (La Tienda)	
■ Compra de objetos (power-ups y cosméticos)	
• Perfil del Piloto y Personalización	
■ Equipamiento de habilidades pasivas y cosméticos	
■ Modificación de credenciales (Nombre y Contraseña) y avatares	
• Salón de la Fama (Rankings)	
■ Consulta de la clasificación global y perfiles	
6. Manual de Aplicación	34
7. Conclusión	38

Propuesta de proyecto

1. IDENTIFICACIÓN DEL PROYECTO:

- **CICLO FORMATIVO:** Desarrollo de Aplicaciones Web (DAW)
- **TÍTULO DEL PROYECTO:** FastFingers
- **DESCRIPCIÓN GENERAL:** Mi proyecto consistiría en un videojuego basado en la mecanografía y la ortografía, en el que varios jugadores compiten entre sí en cada partida, escribiendo correctamente y/o con el menor número de errores ortográficos y tipográficos las frases (generadas por IA, haciendo que cada partida sea completamente diferente a la anterior) que aparecen en la pantalla. El objetivo del proyecto sería educativo y lúdico, ya que a la vez que mejorar la velocidad de escritura de los jugadores se hace a través de métodos gamificados.
- **ALUMNO:** Jorge Barriga Rubio
- **TELÉFONO:** 722492762
- **Fecha presentación:** 31/10/2025
- **E-MAIL:** jorbarrigar@gmail.com

2. OBJETIVO:

El objetivo del proyecto sería desarrollar la web de forma correcta, sin errores, y que a su vez esta cumpla con las expectativas del proyecto. Ha de ser divertida y didáctica. Se implementarían mecánicas battle royale, generando mayor competitividad entre los jugadores, así como rankings, niveles, logros y estadísticas. También se implementarían habilidades, pudiendo el jugador elegir estas a lo largo de su partida y otorgando así diferentes ventajas respecto a los demás. Cada jugador tendrá sus habilidades, no superarán un total de 2, se podrá elegir una cuando se cumpla un minuto después de haber elegido la anterior. Las partidas durarán 3 minutos.

Otro de los objetivos del proyecto es que tenga una interfaz moderna, responsiva y moderna, además de intuitiva. Favoreciendo así la experiencia del usuario y la manejabilidad del sitio.

La correcta incorporación de la inteligencia artificial es otro de los objetivos, haciendo que esta genere las frases de forma dinámica y eficiente.

3. JUSTIFICACIÓN:

La mecanografía es una parte fundamental de la informática y el día a día. Tener una buena velocidad de escritura nos puede ayudar a ser más eficientes y ahorrarnos tiempo. Generalmente, los ejercicios tradiciones de práctica de mecanografía suelen ser aburridos y monótonos, así que, con mi proyecto busco una forma alternativa y motivadora para practicar esta disciplina.

4. ASPECTOS PRINCIPALES QUE SE PRETENDEN ABORDAR:

- Integración justificada y coherente de la inteligencia artificial a través de API.
- Base de datos completa y funcional, que recoja datos de usuarios, puntuaciones y estadísticas.
- Detección automática de errores comparando caracteres.
- Buena documentación del proyecto.
- Implementación de vidas, habilidades y logros.
- Diseño responsive.

5. MEDIOS QUE SE UTILIZAN:

Me gustaría utilizar lenguajes conocidos, como Java, HTML, CSS y JavaScript, pero no descarto usar algún framework o PHP. En cuanto a entornos de desarrollo, los que utilizaré serán VS Code y Eclipse, además de la consola del navegador para realizar trazas y depurar el código. En cuanto a la IA que utilizaré, utilizaré Gemini o ChatGPT.

6. ÁREAS DE TRABAJO Y OTROS ELEMENTOS ESTABLECIDOS POR EL EQUIPO DOCENTE DEL CICLO FORMATIVO A EFECTOS DE VALORACIÓN DE LA PROPUESTA:

- Programación web cliente y servidor.
- Gestión de bases de datos.
- Diseño y usabilidad de interfaces.
- Integración de servicios externos (APIs de IA).
- Control de versiones (GitHub).
- Documentación técnica y memoria del proyecto.
- Pruebas, validación y despliegue final.

Introducción

1. Ficha del proyecto

- **Título del Proyecto:** FastFingers: Plataforma de Mecanografía Gamificada y Competitiva.
- **Autor:** Jorge Barriga Rubio
- **Ciclo Formativo:** Grado Superior en Desarrollo de Aplicaciones Web (DAW)
- **Tutora:** María Carmen Moreno Cortés
- **Año Académico:** 2025/2026

El presente documento detalla el diseño, desarrollo e implementación de **FastFingers**, una plataforma web interactiva orientada al aprendizaje y perfeccionamiento de la mecanografía. A diferencia de los enfoques tradicionales, **FastFingers** se ha concebido como una **Single Page Application** (SPA) inmersiva, con una estética visual inspirada por la estética de la **velocidad y el frenetismo** del *motorsport*, principalmente de la *F1*. La plataforma

transforma el entrenamiento mecanográfico en una **experiencia lúdica y competitiva**.

El proyecto abarca el **desarrollo Full-Stack completo**. En el *Frontend*, se ha prescindido de *frameworks* pesados en favor de **Vanilla JavaScript** para garantizar un rendimiento óptimo y un control granular del Modelo de Objetos del Documento (**DOM**). En el *Backend*, se ha implementado una moderna Arquitectura de Microservicios basada en el ecosistema **Spring Boot y Spring Cloud (Java)**, integrando capacidades avanzadas como comunicaciones bidireccionales en tiempo real mediante **WebSockets** e **Inteligencia Artificial** generativa para la **creación dinámica de contenidos**.

En definitiva, la concepción y ejecución de **FastFingers** representa la **culminación práctica** de los conocimientos adquiridos a lo largo del ciclo formativo. El proyecto trasciende la creación de una simple herramienta educativa para consolidarse como un **Producto Mínimo Viable (MVP)** de nivel profesional. Para ello, se han aplicado **rigurosos estándares de la industria**, incluyendo **patrones de diseño limpios**, seguridad multicapa y un sistema de tolerancia a fallos centralizado, garantizando así un **producto resiliente y preparado** para las exigencias del **entorno tecnológico actual**.

2. Objetivos del proyecto

La creación de **FastFingers** responde a una serie de metas tanto desde la perspectiva del usuario final como desde la exigencia técnica del ciclo formativo:

Objetivos Funcionales (Orientados al usuario):

- **Gamificar el aprendizaje:** Sustituir la monotonía de los ejercicios de mecanografía por dinámicas de juego (recompensas, niveles, progresión) para aumentar la retención y la motivación del usuario.

- **Fomentar la competitividad sana:** Implementar un sistema de clasificación global y modos de juego enfrentados que impulsen a los usuarios a mejorar su agilidad mental y velocidad de tecleo.
- **Ofrecer contenido infinito:** Eliminar la dependencia de bases de datos estáticas mediante la generación procedimental de ejercicios adaptados a los intereses del usuario.

Objetivos Técnicos (Orientados al desarrollo):

- **Diseñar una arquitectura escalable:** Desplegar una infraestructura orientada a microservicios que permita aislar responsabilidades (autenticación, lógica de juego, integración de IA).
- **Dominar la asincronía y el tiempo real:** Implementar un motor de juego en el cliente capaz de sincronizarse a nivel de milisegundos con un servidor y otros clientes mediante WebSockets (STOMP).
- **Demostrar autonomía tecnológica:** Desarrollar el sistema completo sin la dependencia estricta de abstracciones de terceros (como React o Angular en el *Frontend*), gestionando manualmente el estado de la aplicación y la inyección de componentes en el DOM.
- **Desarrollar soltura integrando IA para la generación de datos dinámicos:** Incorporar y consumir de forma eficiente APIs de Modelos de Lenguaje Grande (LLMs), como Google Gemini, para sustituir el contenido predefinido por flujos de datos en tiempo real, demostrando la capacidad de adaptar proyectos de software a las últimas tendencias y demandas del sector tecnológico.

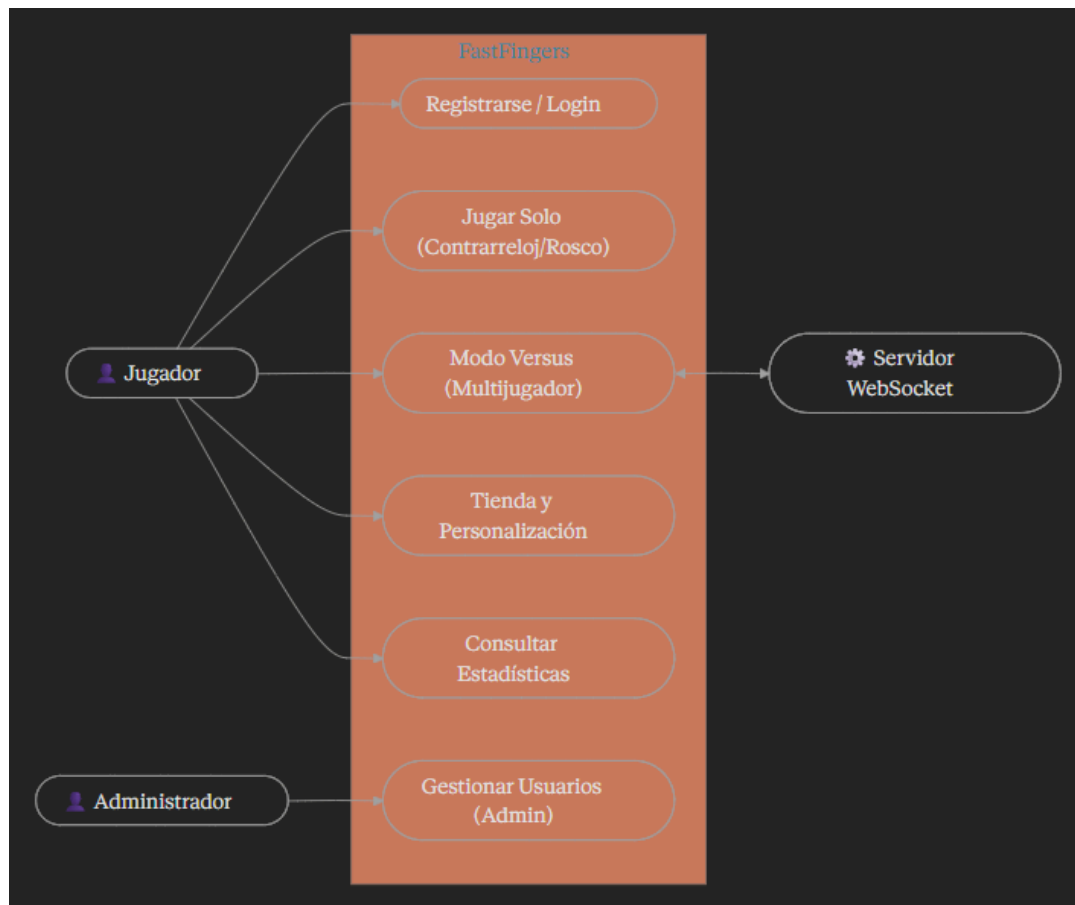
3. Funcionalidades principales

El sistema FastFingers ofrece un conjunto robusto de características y funcionalidades, estructuradas en los siguientes bloques:

- **Gestión de Usuarios y Seguridad:** Sistema de registro y autenticación basado en *JSON Web Tokens* (JWT), con roles definidos (Usuario/Administrador) y cambio de contraseñas.
- **Motor de Partidas Individuales (IA):** Modos de juego "Contrarreloj" y "Partida a Medida", donde el sistema consume la API de Google Gemini para generar textos y "Roscos" léxicos únicos basados en las preferencias y el tema introducido por el jugador.
- **Modo Multijugador (Online):** Creación de salas privadas con códigos de acceso generados dinámicamente, permitiendo a dos usuarios competir en tiempo real en la misma prueba y determinando al final un ganador.
- **Sistema de Economía y Tienda:** Obtención de créditos virtuales según el rendimiento (Palabras acertadas en el rosco y precisión). Estos créditos permiten adquirir "Power-Ups" (ventajas tácticas como congelar el tiempo) y elementos cosméticos, como temas que cambian la interfaz por completo.
- **Garaje y Personalización:** Interfaz para que el usuario gestione su inventario, equipe habilidades pasivas, modifique la interfaz de usuario (temas visuales) y genere y/o modifique su avatar único consumiendo la API de *DiceBear*.
- **Telemetría y Rankings:** Almacenamiento estadístico de las partidas, generación de informes de rendimiento y visualización de un "Salón de la Fama" global, viendo estadísticas de usuarios y créditos de los mismos.
- **Panel de Administración y Subsistema de Logs:** Módulo de control de acceso restringido (solo para administradores) para la gestión de usuarios, roles y exportación de datos a formato Excel. A nivel de *backend*, el sistema audita silenciosamente todas las transacciones mediante Programación Orientada a Aspectos (AOP), capturando

cualquier excepción global y registrando la traza del error exacta en la base de datos sin interrumpir el flujo del cliente.

Para clarificar el funcionamiento del sitio web, aquí tenemos un diagrama que representa los casos de uso de la aplicación, detallando en él las interacciones de los actores con el sistema:



4. Justificación

La pertinencia y necesidad del desarrollo de FastFingers se asienta sobre dos pilares fundamentales: el valor del producto en el sector del software educativo y la ambición arquitectónica desde la perspectiva de la ingeniería de software.

Justificación Pedagógica y de Producto en el ámbito del *EdTech* (tecnología educativa), el entrenamiento mecanográfico tradicional adolece de una altísima tasa de abandono debido a su carácter repetitivo y mecánico. La aplicación profunda de técnicas de **gamificación** resuelve este obstáculo al introducir bucles de retroalimentación positiva que mantienen la atención del usuario.

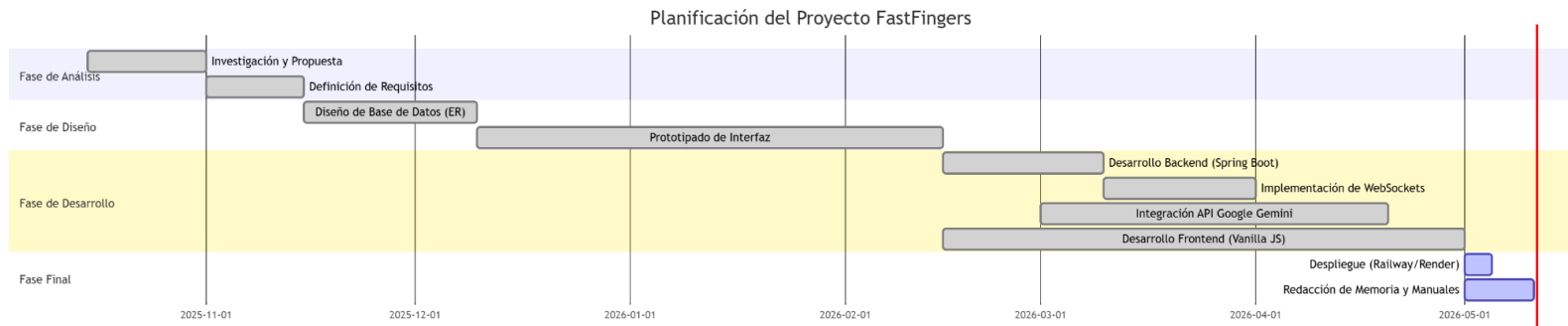
La inclusión de elementos propios del diseño de videojuegos —como un "Salón de la Fama" global, enfrentamientos en vivo (*matchmaking*) y mecánicas de riesgo-recompensa mediante la activación estratégica de objetos durante la prueba— enmascara el esfuerzo académico bajo un entorno puramente lúdico.

Justificación Tecnológica y Académica desde el punto de vista informático, la elección de este proyecto constituye un reto de alto calibre que valida de forma integral las competencias adquiridas en el ciclo de Desarrollo de Aplicaciones Web. A diferencia de las aplicaciones monolíticas de gestión convencionales (típicos sistemas CRUD de altas y bajas), FastFingers requiere resolver problemas complejos de concurrencia y sincronización de estados distribuidos.

La decisión de abandonar el monolito en favor de una **Arquitectura de Microservicios** (orquestrada mediante un *API Gateway* y un servidor de descubrimiento *Eureka*) no solo asegura un entorno de alta disponibilidad y escalabilidad, sino que demuestra al tribunal la capacidad de diseñar soluciones empresariales modernas.

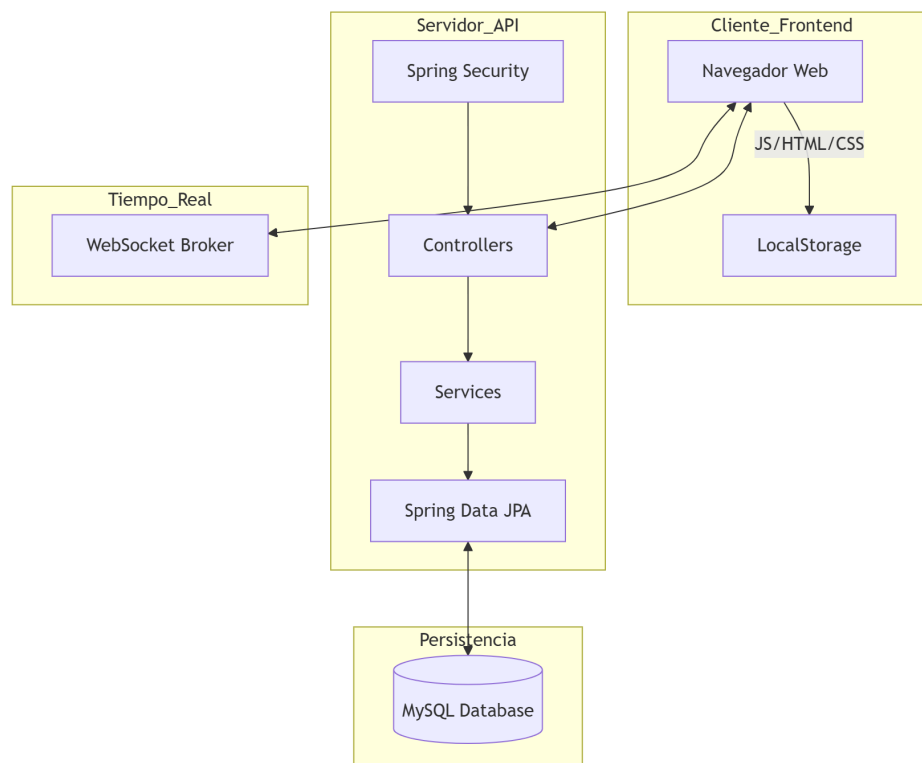
En el *Frontend*, la elección deliberada de prescindir de *frameworks* abstractos (React o Angular) a favor de *Vanilla JavaScript* modular persigue un objetivo claro: demostrar un dominio absoluto del lenguaje base, la manipulación directa del DOM y la gestión manual del estado. Finalmente, la integración de *Large Language Models* (LLMs) sitúa al proyecto a la vanguardia tecnológica, incorporando así la Inteligencia Artificial generativa en flujos transaccionales reales, una de las habilidades más demandadas en el panorama laboral actual.

Para garantizar la viabilidad y el cumplimiento de los plazos de entrega, el desarrollo de FastFingers se ha estructurado siguiendo la planificación temporal que se ilustra a continuación:



Diseño

Este es el diagrama de Arquitectura N-Capas, detallando la comunicación entre el Cliente (Frontend), el Servidor de Aplicaciones (Spring Boot) y la Capa de Persistencia (MySQL).



1. Diseño de la base de datos

Para el almacenamiento persistente de la información de FastFingers, se ha optado por un sistema gestor de bases de datos relacional (RDBMS), concretamente **MySQL**. Esta elección se fundamenta en la necesidad de mantener la integridad referencial y la consistencia de los datos en un entorno donde convergen transacciones económicas virtuales (créditos), historiales de partidas y un sistema de control de acceso.

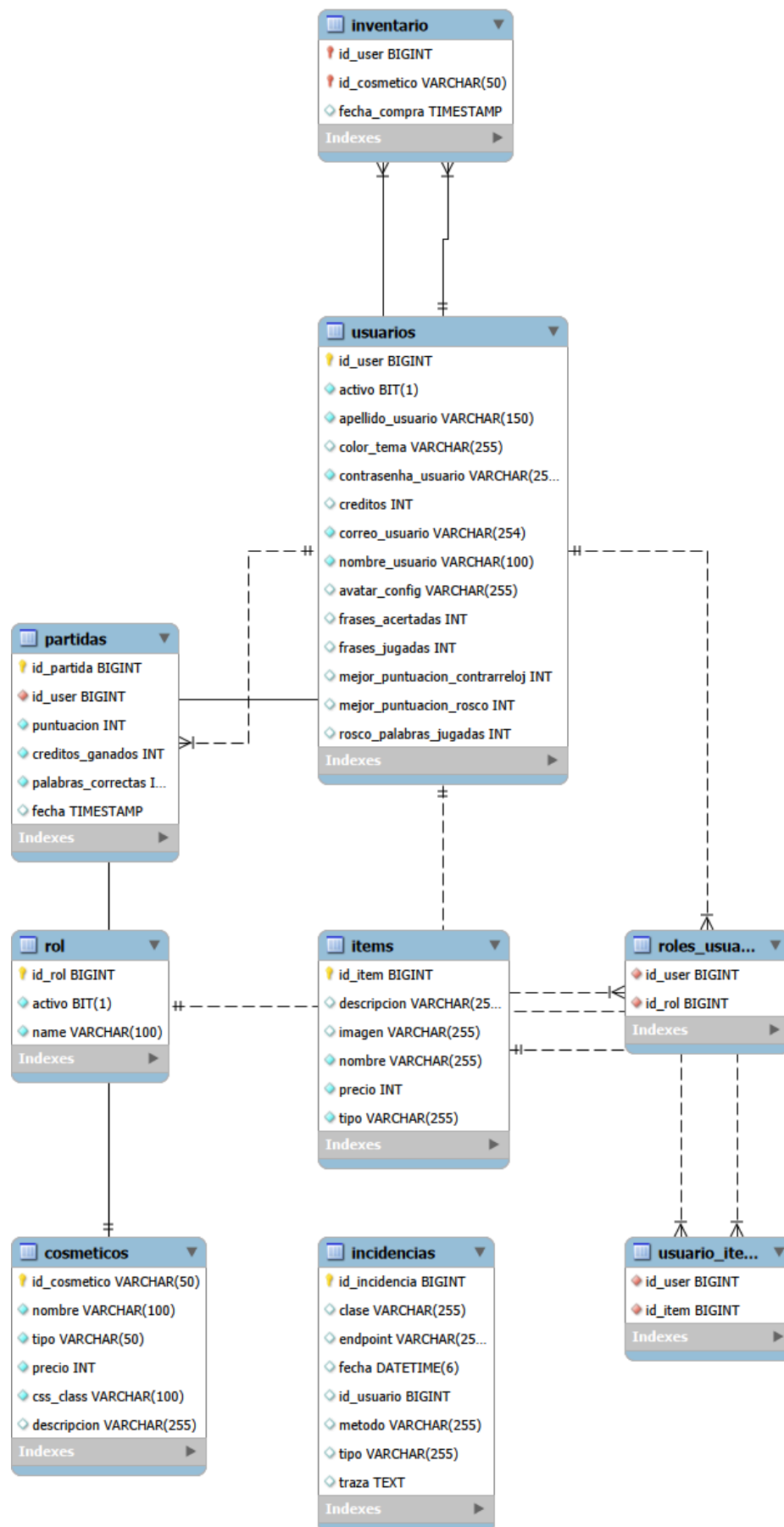
La interacción entre la aplicación y la base de datos se realiza mediante el estándar *Java Persistence API* (JPA) a través del *framework* Spring Data JPA, el cual mapea las entidades orientadas a objetos (ORM) directamente a las tablas relacionales. A continuación, se detalla la estructura del Modelo Entidad-Relación (ER) generado:

- **Entidad Central (usuarios):** Es el núcleo del modelo. Almacena tanto la información de autenticación (correo y contraseña encriptada) como las estadísticas acumuladas del jugador (frases acertadas, jugadas, mejor puntuación en contrarreloj y rosco). Además, incluye campos de personalización como la configuración del avatar (avatar_config) y el tema visual seleccionado (color_tema).
- **Seguridad y Control de Acceso (rol y roles_usuario):** Se implementa un sistema de control de acceso basado en roles (RBAC). La tabla rol define los niveles de privilegios (ej. Piloto, Administrador). La relación de muchos a muchos (N:M) se resuelve mediante la tabla intermedia roles_usuario, permitiendo que un usuario posea múltiples roles simultáneamente.
- **Economía y Equipamiento (items, cosmeticos, usuario_item, inventario):** El sistema económico del juego se divide en dos grandes ramas. Por un lado, los items (objetos tácticos o *Power-Ups* como multiplicadores o congeladores de tiempo), los cuales se asocian al

usuario mediante la tabla puente `usuario_item`. Por otro lado, los cosmeticos (modificaciones visuales para la interfaz), almacenados en un catálogo propio y vinculados al jugador mediante la tabla intermedia inventario, la cual registra el momento exacto de la adquisición mediante un *timestamp*.

- **Telemetría y Rendimiento (partidas):** Entidad fundamental para el "Salón de la Fama" y la progresión del usuario. Registra una relación de uno a muchos (1:N) con el usuario, almacenando cada intento de juego junto con su puntuación, palabras correctas, créditos obtenidos y la fecha de la ejecución.
- **Auditoría y Soporte (incidencias):** Tabla aislada del flujo de juego principal, diseñada específicamente para el microservicio de control de errores. Registra de forma automática cualquier excepción generada en el *Backend*, almacenando el *endpoint*, la clase, el método, el tipo de error y la traza completa, facilitando así la depuración sin exponer información sensible al cliente.

A continuación se puede ver el diagrama entidad-relación de la base de datos, sacado directamente de MySQL:



2. Diseño del sitio web

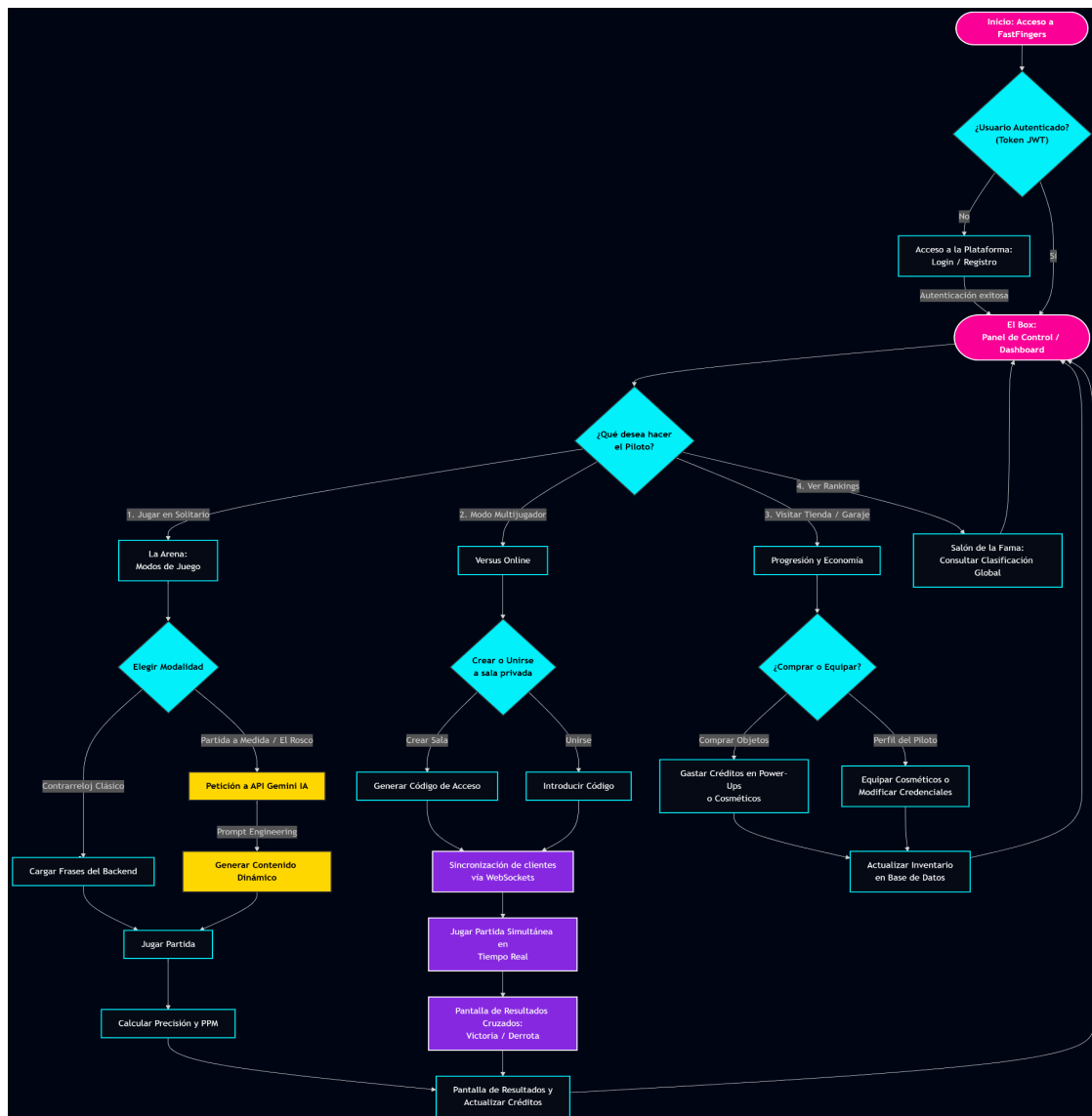
El diseño de la interfaz de usuario (UI) y la experiencia de usuario (UX) de FastFingers rompe con los esquemas estáticos y formales de las herramientas educativas tradicionales. Se ha concebido bajo una fuerte **estética Cyberpunk y de "Racing" (carreras)**, utilizando el alto contraste y los elementos dinámicos para adaptar al jugador en un entorno competitivo.

Los principios de diseño y maquetación aplicados son los siguientes:

- **Arquitectura Single Page Application (SPA):** La plataforma opera bajo un único archivo `index.html`. A través de *Vanilla JavaScript*, se manipula el Modelo de Objetos del Documento (DOM) para inyectar, ocultar y mostrar distintas "pantallas" o vistas (Login, Menú Principal, Arena de Juego, Garaje, Tienda) sin necesidad de recargar la página. Esto elimina la latencia de navegación y proporciona una sensación de fluidez similar a la de un videojuego nativo.
- **Identidad Visual y Paleta de Colores:** Se ha definido un sistema de variables CSS globales (`:root`) para garantizar la consistencia visual. El fondo principal emula una textura de fibra de carbono oscura (`#050b14`), sobre la cual destacan colores neón vibrantes: el Cian eléctrico (`#00f2fe`) como color principal, el Magenta (`#fe019a`) para los elementos secundarios o rivales en el modo multijugador, y el Dorado (`#ffd700`) para el sistema de economía y recompensas.
- **Tipografía y Dinamismo:** Para reforzar la sensación de velocidad, se ha empleado la familia tipográfica *Montserrat* en sus variantes de mayor grosor (*Black* o *ExtraBold*) y en formato cursiva (*italic*). Los elementos interactivos, como botones y contenedores de datos, aplican transformaciones de sesgo (`skewX(-12deg)`) emulando las formas aerodinámicas del mundo del motor.

- **Diseño *Responsive* y *Mobile-First*:** A pesar de la complejidad de la interfaz (cronómetros, barras de progreso y modales superpuestos), la maquetación se ha estructurado utilizando *CSS Flexbox* y *CSS Grid*. Mediante *Media Queries*, el sistema detecta la resolución del dispositivo y reorganiza las columnas de información, apilando los menús verticalmente en terminales móviles para asegurar que la usabilidad táctil sea óptima y no existan desbordamientos horizontales.

Flujo de Usuario (User Journey y UX): Antes de definir la capa visual, se ha modelado el comportamiento del sistema para estructurar un árbol de navegación coherente. Dado que la plataforma opera como una *Single Page Application* (SPA), es vital evitar "callejones sin salida". El siguiente diagrama ilustra el ciclo de vida completo de una sesión, abarcando desde la validación de seguridad (JWT) hasta la bifurcación hacia los tres pilares del sistema: la arena de juego (IA y WebSockets), la tienda virtual y el Salón de la Fama.



Aspectos interesantes del código

El desarrollo de FastFingers ha requerido abordar desafíos técnicos que trascienden el típico desarrollo web de aplicaciones CRUD (Crear, Leer, Actualizar, Borrar). Para garantizar la escalabilidad, resiliencia y el rendimiento en tiempo real, se

han implementado diversos patrones de diseño y arquitecturas avanzadas. A continuación, se destacan los fragmentos más críticos del sistema:

- **Ingeniería de Instrucciones (Prompt Engineering) y Tolerancia a Fallos**

La generación dinámica de contenido delegada a la Inteligencia Artificial (Google Gemini) presentaba el riesgo de recibir respuestas no estructuradas que corrompieran la interfaz del usuario. Para solucionarlo, en el microservicio correspondiente, se ha implementado un estricto control del *Prompt* y un mecanismo de *Fallback*.

```

public String generarFrase(String dificultad, String temaPersonalizado) {
    String url = "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.1-flash-lite-
    preview:generateContent?key=" + apiKey;

    String temaFinal = "";

    if (temaPersonalizado != null && !temaPersonalizado.trim().isEmpty()) {
        temaFinal = temaPersonalizado.trim();
    } else {
        String[] temas = {
            "la antigua Roma", "el espacio exterior", "los animales marinos",
            "los inventos tecnológicos", "la naturaleza", "la gastronomía mundial",
            "la inteligencia artificial", "el cuerpo humano", "la historia del arte",
            "los dinosaurios", "la física cuántica", "mitología griega", "historia del fútbol",
            "formula 1", "nba", "juegos olímpicos"
        };
        // 3. Usamos nextInt() para coger un número entero limpio
        temaFinal = temas[random.nextInt(temas.length)];
    }

    String prompt = "Escribe 5 frases curiosas sobre " + temaFinal + ". " +
        "Usa una gramática natural y fluida de 10 a 15 palabras por frase. " +
        "REGLA Estricta: Responde ÚNICAMENTE con las frases separadas por el carácter '|'. " +
        "No incluyas saludos ni repitas instrucciones. " +
        "Al final de todo el bloque, añade el símbolo '#'.";

    String requestBody = "{" +
        "\"contents\": [{\"parts\": [{\"text\": \"" + prompt + "\"}]}],\" +
        "\"generationConfig\": {\" +
        "\"temperature\": 0.7,\" +
        "\"maxOutputTokens\": 1500\" +
        "}" +
        "};";

    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_JSON);
    HttpEntity<String> entity = new HttpEntity<>(requestBody, headers);

    try {
        ResponseEntity<String> response = restTemplate.postForEntity(url, entity, String.class);

        JsonNode root = objectMapper.readTree(response.getBody());
        String fraseGenerada = root.path("candidates").get(0)
            .path("content")
            .path("parts").get(0)
            .path("text").asText().trim();

        // 4. Usamos logger.info en lugar de System.out.println
        logger.info("TEMA ELEGIDO PARA ESTA TANDA: {}", temaFinal);

        return fraseGenerada;
    } catch (Exception e) {
        // 5. Usamos logger.error en lugar de System.err.println
        logger.error("Fallo en boxes al contactar con Gemini: {}", e.getMessage());
        return "El servidor de IA está repostando combustible|Inténtalo de nuevo en unos segundos|El coche de seguridad está en pista|";
    }
}

```

Como se observa en el código superior, se fuerza a la IA a devolver un formato predecible (separado por caracteres delimitadores o formato JSON). Posteriormente, el sistema *parsea* la respuesta. El aspecto arquitectónico más relevante reside en el bloque catch: si la API externa experimenta latencia extrema o agota su cuota de peticiones, el servidor inyecta silenciosamente una batería de frases predefinidas. Esto garantiza la disponibilidad del juego; el usuario final nunca percibe la caída del servicio externo.

- **Comunicaciones Full-Duplex mediante WebSockets**

El modo multijugador ("Versus") descartó el protocolo HTTP tradicional debido a su sobrecarga e ineficiencia para retransmitir estados de progreso en milisegundos. En su lugar, se implementó el protocolo STOMP sobre WebSockets.

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

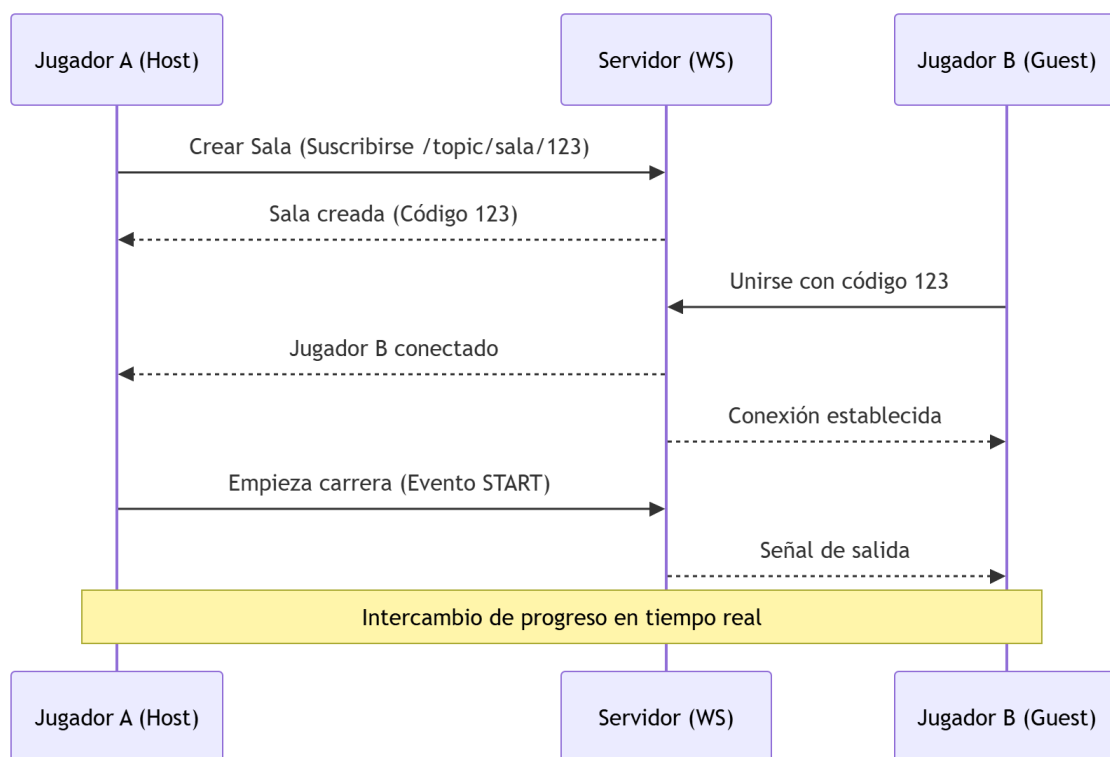
    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        // El prefijo para los mensajes que el servidor ENVÍA a los clientes (Ej: a la sala de juego)
        config.enableSimpleBroker("/topic");
        // El prefijo para los mensajes que los clientes ENVÍAN al servidor
        config.setApplicationDestinationPrefixes("/app");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // La URL a la que se conectará el JavaScript para abrir el túnel
        registry.addEndpoint("/ws-game")
            .setAllowedOriginPatterns("*") // Permite que cualquier frontend se conecte
            .withSockJS(); // Fallback de seguridad por si algún navegador antiguo falla
    }
}
```

El modo multijugador ("Versus") descartó el protocolo HTTP tradicional debido a su sobrecarga e ineficiencia para retransmitir estados de progreso en milisegundos. En su lugar, se implementó el protocolo STOMP sobre WebSockets.

El servidor Spring Boot actúa como un Message Broker, permitiendo a los clientes suscribirse a salas dinámicas. Esto no solo sincroniza la cuenta atrás inicial al milisegundo, sino que permite retransmitir las pulsaciones de teclas y aciertos. Además, la gestión de conexiones está diseñada para ser resiliente ante caídas de red abruptas: si un usuario pierde la conexión, el evento `SessionDisconnectEvent` intercepta la salida y libera los recursos individuales de ese socket para evitar fugas de memoria. Sin embargo, **el servidor mantiene activa la sala virtual**, garantizando la persistencia del estado de la partida para que el jugador restante no sufra interrupciones, pueda finalizar la partida con normalidad y reclamar su victoria.

Mediante el siguiente diagrama de secuencia se clarifica esta conexión:



- **Intercepción y Auditoría mediante Programación Orientada a Aspectos (AOP)**

En un ecosistema de microservicios, el rastreo de errores puede convertirse en una tarea inmanejable si se programa de forma imperativa clásica. Para mantener el código de negocio limpio (Principio de Responsabilidad Única), se aplicó el paradigma AOP.

```
@Aspect
@Component
public class LoggingAspect {

    //logger de SLF4J
    private static final Logger log = LoggerFactory.getLogger(LoggingAspect.class);

    @Pointcut("within(com.jorge.sprintdef.services..*)")
    public void vigilarServicios() {}

    @Before("vigilarServicios()")
    public void logAntesDeEntrar(JoinPoint joinPoint) {
        log.info("Ejecutando: {}", joinPoint.getSignature().getName());
    }

    @AfterReturning(pointcut = "vigilarServicios()", returning = "resultado")
    public void logDespuesDeSalir(JoinPoint joinPoint, Object resultado) {
        String nombreMetodo = joinPoint.getSignature().getName();

        if (resultado instanceof String string) {
            String respuesta = string.toLowerCase();

            if (respuesta.contains("error") || respuesta.contains("no encontrado") ||
                respuesta.contains("no válido")) {
                String mensaje = "Rechazado en " + nombreMetodo + " | Motivo: " + resultado;
                log.warn("[AVISO_NEGOCIO] {}", mensaje);
                return;
            }
        }

        log.info("Finalizado con éxito: {}", nombreMetodo);
    }

    @AfterThrowing(pointcut = "vigilarServicios()", throwing = "error")
    public void logEnCasoDeError(JoinPoint joinPoint, Throwable error) {
        String mensaje = "Fallo en: " + joinPoint.getSignature().getName() + " | Causa: " +
            error.getMessage();
        String tipoError = "ERROR_SISTEMA";

        if (error instanceof NotFoundException) tipoError = "ERROR_NO_ENCONTRADO";
        else if (error instanceof DuplicateException) tipoError = "ERROR_DUPLICADO";
        else if (error instanceof BadRequestException) tipoError = "ERROR_PETICION_INVALIDA";

        log.error("[{}] {}", tipoError, mensaje);
    }
}
```

Mediante la definición de *Pointcuts* (puntos de corte), el sistema intercepta automáticamente la entrada y salida de todos los métodos dentro de la capa de

servicios. Destaca especialmente el uso de `@AfterThrowing`, que captura de forma global cualquier excepción lanzada. De este modo, la auditoría y persistencia de las incidencias operan como una capa transversal e invisible.

- **API Gateway y Seguridad Perimetral**

Al desplegar múltiples microservicios (Usuarios, Incidencias, etc.), el *Frontend* se enfrentaba a problemas complejos de seguridad, enrutamiento cruzado y políticas del navegador (CORS). Se resolvió implementando un punto único de entrada.

```
Configuration
public class GatewayRoutesConfig {

    @Value("${URI_USUARIOS:http://localhost:8081}")
    private String uriUsuarios;

    @Value("${URI_INCIDENCIAS:http://localhost:8082}")
    private String uriIncidencias;

    @Bean
    public RouteLocator customRouteLocator(RouteLocatorBuilder builder) {
        return builder.routes()
            .route("usuarios-service", r -> r.path(
                "/auth/**",
                "/usuarios/**",
                "/roles/**",
                "/usuarios_rol/**",
                "/ws-game/**"
            )
            // Eliminamos los CORS duplicados
            .filters(f -> f
                .dedupeResponseHeader("Access-Control-Allow-Origin", "RETAIN_UNIQUE")
                .dedupeResponseHeader("Access-Control-Allow-Credentials",
"RETAIN_UNIQUE")
            )
            .uri(uriUsuarios))
            .route("incidencias-service", r -> r.path("/incidencias/**")
            // También se lo ponemos a incidencias por si en el futuro pasa lo mismo
            .filters(f -> f
                .dedupeResponseHeader("Access-Control-Allow-Origin", "RETAIN_UNIQUE")
                .dedupeResponseHeader("Access-Control-Allow-Credentials",
"RETAIN_UNIQUE")
            )
            .uri(uriIncidencias))
            .build();
    }
}
```

El *API Gateway* aísla la red interna. El cliente JavaScript jamás conoce la IP o el puerto real de los microservicios finales. Además de balancear la carga consultando al servidor de descubrimiento (Eureka), el Gateway actúa como

escudo perimetral, inyectando y limpiando las cabeceras Access-Control-Allow-Origin para satisfacer las medidas de seguridad del navegador, evitando configuraciones redundantes en cada microservicio individual.

Manual de usuario

- **Acceso a la Plataforma (Primeros Pasos)**

El acceso a FastFingers está diseñado para ser rápido e intuitivo, asegurando que los datos de cada piloto estén protegidos y su progreso se guarde en la nube de forma segura.

- **Registro de Nueva Cuenta**

Para los nuevos pilotos, el primer paso es darse de alta en el sistema.

- Haz clic en el botón **"Registrarse"** en la pantalla de bienvenida.
- Completa el formulario con tu Nombre, Apellidos, Correo Electrónico y una Contraseña segura.
- El sistema validará en tiempo real que el correo no esté en uso. Al confirmar, tu perfil se creará con los marcadores a cero y serás redirigido al panel principal.

CREAR CUENTA

prueba

probando

test@gmail.com

....

REGISTRARSE

¿Ya tienes cuenta? [Inicia sesión](#)

VOLVER A LA PÁGINA PRINCIPAL

- **Inicio de Sesión**

Si ya tienes cuenta, la autenticación es instantánea.

- Introduce tu correo y contraseña en la pantalla de **Login**. Al pulsar entrar, el sistema generará un token de sesión seguro, cargará tu inventario, tus créditos y tus estadísticas históricas en milisegundos sin necesidad de recargar la página web.



- **El Box (Panel de Control Principal)**

Una vez iniciada la sesión, el jugador accede a "El Box", el centro de mandos o *Dashboard* de FastFingers.

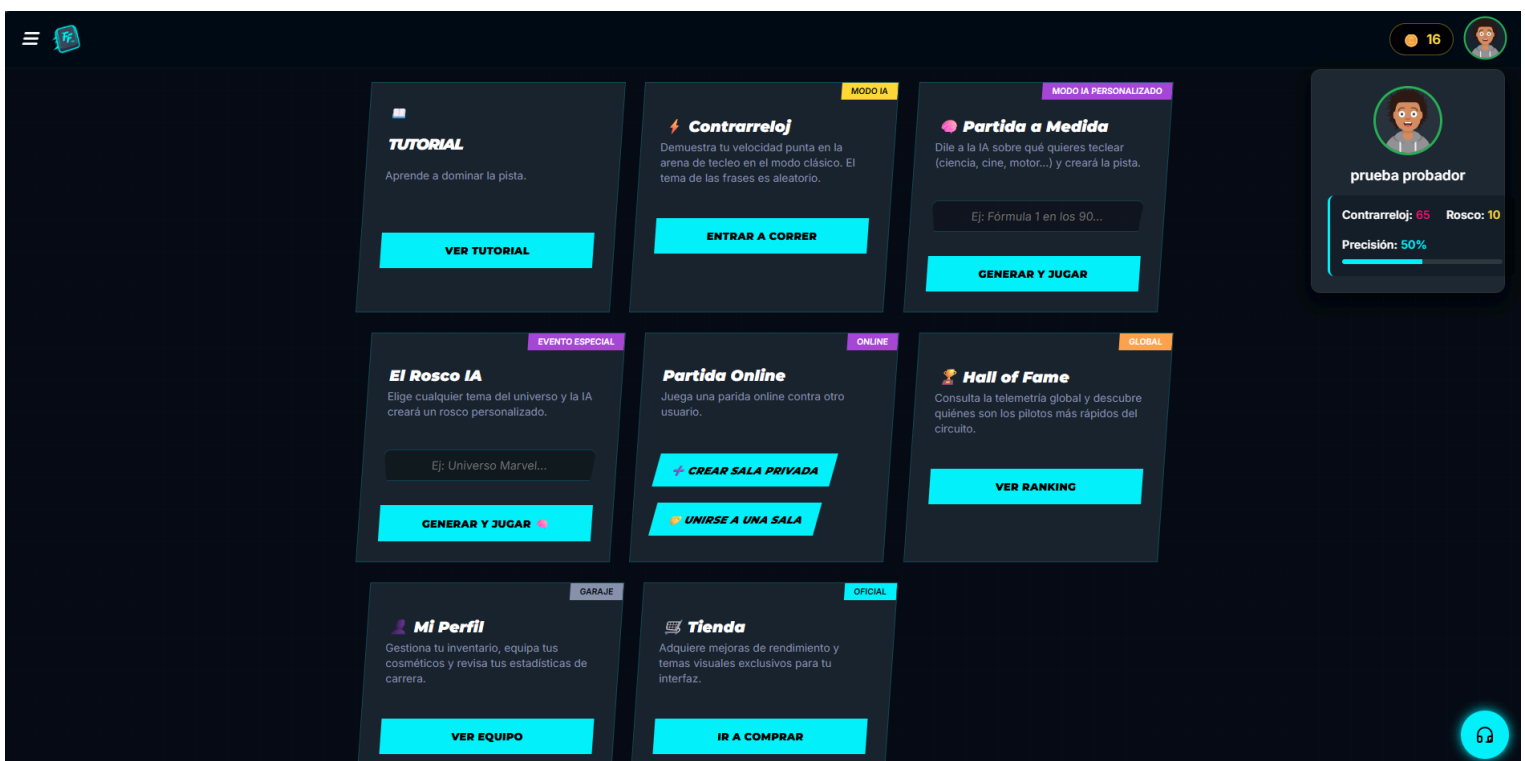
- **Navegación y HUD**

Desde aquí, el usuario tiene acceso a todas las secciones mediante el menú lateral izquierdo. En la esquina superior derecha siempre se mostrará el Avatar del jugador, su nombre y su saldo actual de Créditos.

- **Telemetría y Asistencia**

Justo debajo del perfil, el jugador dispone de un panel analítico en tiempo real que muestra los siguientes datos:

- **Contrarreloj:** Récord máximo del modo Contrarreloj/Partida a Medida.
- **El Rosco:** Mejor puntuación en este modo.
- **Precisión:** Una barra de progreso visual que mide el porcentaje de acierto global.
- **Asistente de Boxes:** En la esquina inferior derecha, el usuario dispone de un Chatbot interactivo (icono de mensaje) que resuelve dudas frecuentes sobre el juego al instante.



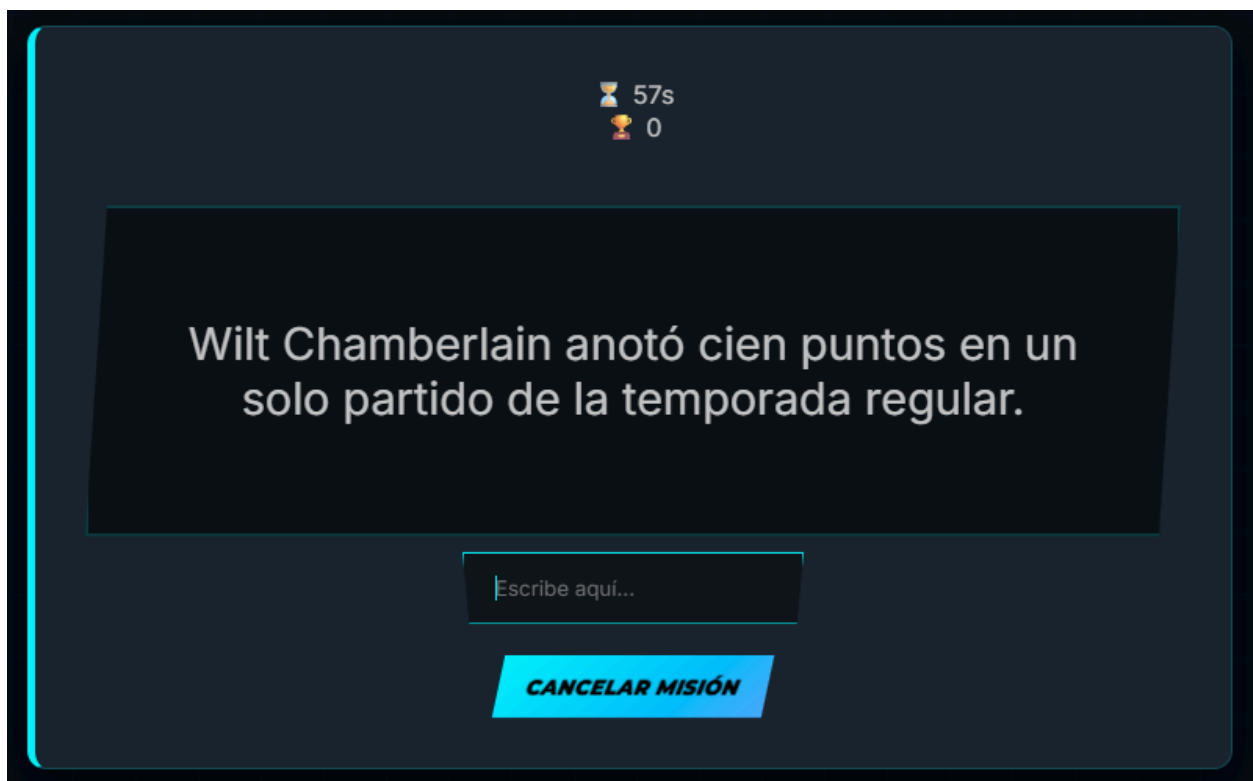
- **La Arena (Modos de Juego Un Jugador)**

La Arena es el campo de entrenamiento donde los pilotos consiguen sus recursos.

- **Contrarreloj Clásico**

Es la prueba de fuego de la plataforma.

- Al acceder, el semáforo marcará el inicio.
- El usuario debe teclear la frase que aparece en pantalla con la mayor precisión posible. Cada pulsación correcta suma puntos al combo; un error rompe la racha y obliga a corregir.
- El HUD muestra el tiempo restante, la precisión actual y las palabras por minuto (PPM) en tiempo real.



- **Partida a Medida (IA Generativa)**

Esta funcionalidad exclusiva permite generar un reto único.

- El usuario introduce una temática en el cuadro de texto (Ej: *"Historia de la informática"* o *"Fórmula 1"*).
- El sistema se comunica con la Inteligencia Artificial para generar un texto coherente e inédito sobre esa temática.
- Tras unos segundos de generación, el jugador mecanografía un texto que no existe en ninguna base de datos predefinida.



- **El Rosco**

Un desafío de **resistencia mental y agilidad**.

- Se presenta una interfaz circular con **25 letras del alfabeto**.
- Para cada letra, el jugador debe **completar una palabra** antes de que el tiempo se agote.
- Es ideal para **practicar transiciones rápidas** en el teclado.



- **Modo Versus (Multijugador Online)**

El núcleo competitivo de FastFingers. Permite carreras de mecanografía en tiempo real contra oponentes de cualquier parte del mundo.

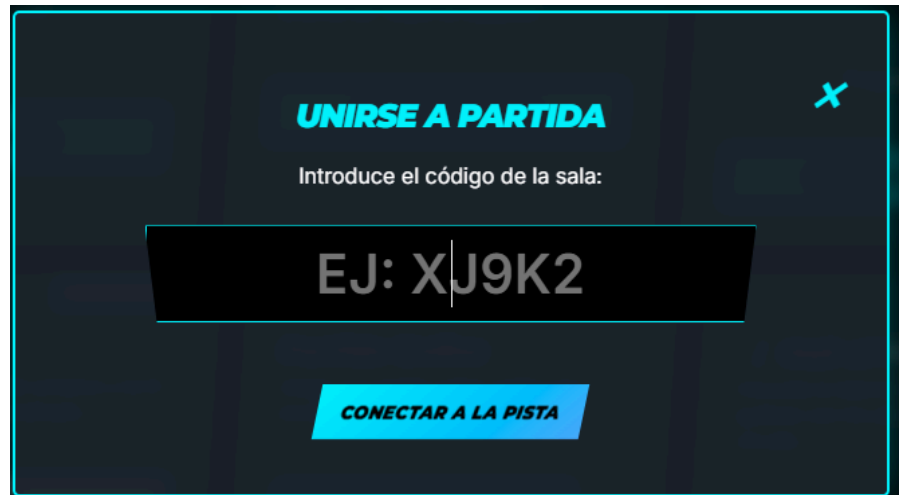
- **Creación de la Sala (El Host)**

- Un jugador entra a la pestaña "Multijugador" y selecciona "**Crear Sala**".
- El sistema genera un código alfanumérico único (Ej:T9Q6R) y deja al jugador en estado de espera.

- **Unirse a la Carrera (El Invitado)**

- El oponente selecciona "**Unirse a Sala**" e introduce el código facilitado por el creador.
- Inmediatamente, la pantalla se divide mostrando las tarjetas y avatares de ambos jugadores frente a frente.

- Una cuenta atrás (3, 2, 1...) da inicio a la partida simultánea.



- **Sincronización y Resultados**

Durante la partida, las barras de progreso de ambos jugadores se actualizan en pantalla en tiempo real. Al finalizar el texto, se declara un vencedor absoluto. El ganador se lleva la recompensa principal en créditos, mientras el perdedor obtiene un pequeño incentivo por participación.

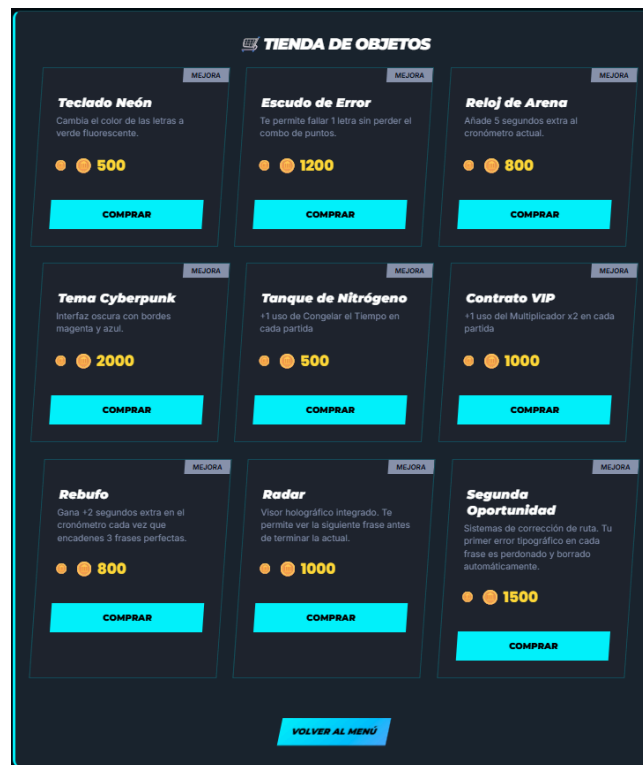
- **El Garaje (La Tienda)**

Todo el esfuerzo en La Arena se materializa en El Garaje, el centro de personalización de la plataforma.

- **Adquisición de Objetos**

Usando los créditos ganados en las partidas, el jugador puede adquirir ventajas para el juego:

- **Power-Ups (Ventajas Tácticas):** Objetos de un solo uso como el *"Reloj de Arena"* (+5 segundos) o el *"Escudo de Error"* (perdona la primera falta).
- **Cosméticos:** Mejoras visuales permanentes como el *"Tema Cyberpunk"*, que cambian drásticamente los colores y la interfaz del juego para adaptarse a los gustos del piloto.
- Para comprar, selecciona un ítem de la vitrina y confirma la operación. El saldo se descontará de forma inmediata y el objeto pasará a tu inventario.



- **Perfil del Piloto y Configuración**

El espacio personal del usuario para gestionar su cuenta y su inventario, pudiendo desde ahí administrar sus objetos y cambiar credenciales de acceso, incluido también el aspecto de su avatar.

- **Gestión de Inventario**

Dentro del Perfil, el usuario puede ver los ítems que ha comprado en la tienda. Mediante un sistema de interruptores rápidos, puede elegir qué Power-Up llevar equipado para su siguiente duelo o qué Tema visual activar.

- **Edición de Credenciales y Avatar**

- **Avatar:** FastFingers permite modificar la imagen del jugador al instante mediante un motor de avatares. Al cambiarlo, se actualiza automáticamente en el ranking global y en las partidas multijugador.
- **Seguridad:** El usuario puede actualizar su contraseña o cambiar su nombre de visualización en cualquier momento desde los ajustes de cuenta.

TU PERFIL

[VOLVER AL MENÚ](#)

[INVENTARIO](#) [AJUSTES](#)

CONFIGURACIÓN DEL PILOTO

Nombre de Usuario
prueba

Correo Electrónico
No disponible

Nueva Contraseña
Dejar en blanco para no cambiar



PELO / SOMBRERO
Pelo Corto Clásico

ACCESORIOS
Ninguno

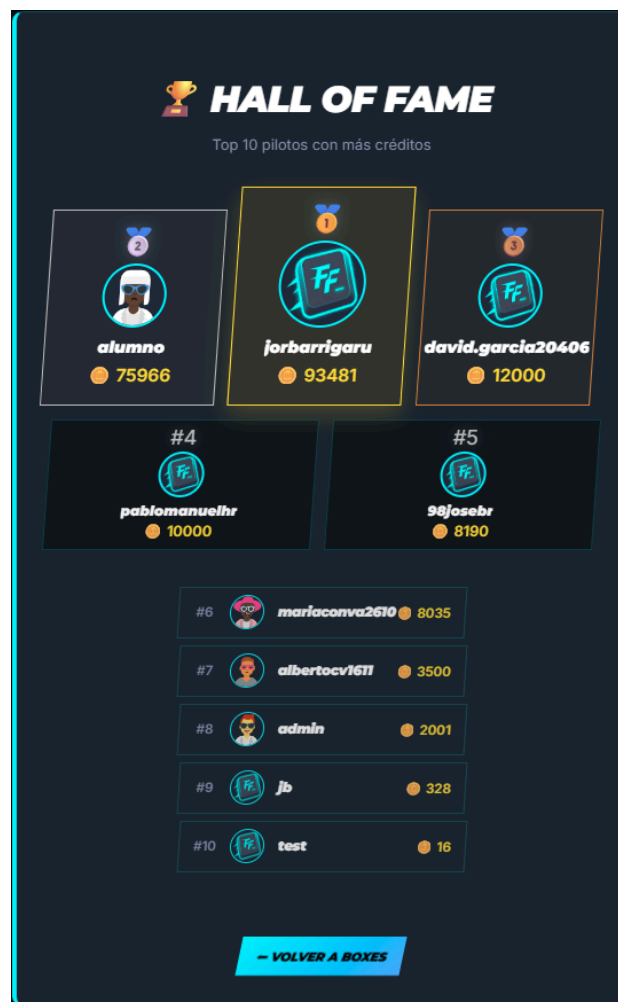
[GUARDAR LOOK](#)

[GUARDAR CAMBIOS](#)

- **Salón de la Fama (Rankings)**

Para los pilotos más competitivos, la pestaña "Ranking" actúa como un registro histórico global.

- **Tabla de Clasificación:** Muestra la lista de los jugadores con más créditos de la plataforma.
- **Inspección de Pilotos:** Al hacer clic en cualquier jugador de la tabla, se despliega una tarjeta holográfica con sus estadísticas detalladas (Récords de modos de juego, precisión y créditos obtenidos), permitiendo analizar el rendimiento de los oponentes.



Manual de aplicación

El presente manual está orientado a desarrolladores y administradores de sistemas encargados de la instalación, configuración y mantenimiento de la plataforma FastFingers. A continuación, se detallan los requisitos técnicos y la arquitectura subyacente.

- **Requisitos del Entorno y Stack Tecnológico**

La aplicación se fundamenta en una arquitectura distribuida y desacoplada, requiriendo los siguientes componentes:

- **Backend (Microservicios):** Desarrollado sobre el ecosistema Spring Boot y Spring Cloud (Java). Requiere un entorno de ejecución Java (JDK).
 - **Frontend (Single Page Application):** Construido íntegramente con Vanilla JavaScript, HTML5 y CSS3, prescindiendo de frameworks externos como React o Angular para un control absoluto del DOM.
 - **Base de Datos:** MySQL Server, operando como sistema gestor de bases de datos relacional (RDBMS).
 - **Integraciones Externas:** Conexión a Internet obligatoria para el consumo de la API de Google Gemini (Inteligencia Artificial generativa) y la API de DiceBear para la generación dinámica de avatares.
- **Configuración de la Base de Datos**

El esquema relacional garantiza la integridad de las transacciones del juego:

- **ORM:** El mapeo de las tablas se realiza de forma automatizada mediante Spring Data JPA e Hibernate.
- **Estructura Principal:** El sistema genera automáticamente las tablas centrales, incluyendo usuarios, tablas de control de acceso (rol,

roles_usuario), economía (items, cosmeticos, inventario, usuario_item), telemetría (partidas) y auditoría (incidencias).

- **Configuración y Despliegue del Backend**

El backend no opera como un monolito, sino bajo una Arquitectura de Microservicios:

- **Orquestación:** Es necesario levantar el servidor de descubrimiento (Eureka) para que los distintos microservicios puedan registrarse y comunicarse entre sí.
- **API Gateway y Seguridad Perimetral:** Todas las peticiones del frontend deben dirigirse al API Gateway. Este componente actúa como escudo perimetral, ocultando las IPs reales de los microservicios y gestionando centralizadamente las políticas de CORS inyectando las cabeceras Access-Control-Allow-Origin.
- **Seguridad:** Las rutas protegidas validan la identidad del usuario mediante JSON Web Tokens (JWT) y un sistema de control de acceso basado en roles (RBAC).
- **Auditoría Silenciosa (AOP):** No es necesario revisar logs de consola continuamente. Mediante Programación Orientada a Aspectos (AOP), el sistema intercepta cualquier excepción (Pointcuts) y registra la traza completa directamente en la tabla incidencias de la base de datos sin interrumpir el flujo del cliente.

- **Generación de Contenido (Inteligencia Artificial)**

Para el modo "Partida a Medida":

- Se requiere configurar la apiKey en las propiedades del servidor para autorizar las peticiones a la API generativelanguage.googleapis.com.
- **Sistema de Fallback:** En caso de que la API de Gemini agote su cuota de peticiones o sufra latencia severa, el servidor inyectará automáticamente un array de frases predefinidas almacenadas localmente, garantizando así la alta disponibilidad del juego.

- **Gestión de WebSockets (Modo Multijugador)**

Para habilitar el modo competitivo en tiempo real:

- **Protocolo:** La comunicación Full-Duplex se realiza utilizando el protocolo STOMP sobre WebSockets.
- **Endpoints y Broker:** El frontend debe establecer la conexión inicial apuntando al endpoint /ws-game. El servidor Spring Boot actúa como Message Broker, gestionando canales de suscripción (/topic) y recepción (/app).
- **Gestión de Conexiones y Memoria:** El sistema cuenta con mecanismos de tolerancia a fallos de red. El evento SessionDisconnectEvent garantiza la recolección de basura (Garbage Collection) de los sockets huérfanos. A nivel lógico, la sesión de la sala se mantiene persistente en la memoria del servidor el tiempo estrictamente necesario para que el cliente rival finalice su transacción.

- **Configuración del Frontend**

El despliegue del cliente es directo al tratarse de archivos estáticos:

- Al operar bajo una arquitectura Single Page Application (SPA), el servidor web (Nginx, Apache, etc.) debe apuntar unívocamente al archivo `index.html`.
- La navegación entre vistas (El Box, La Arena, El Garaje) se gestiona inyectando y ocultando componentes en el DOM de forma dinámica sin recargas de página.

Conclusiones

- **Conclusión y Líneas Futuras**

El diseño, desarrollo y despliegue de la plataforma FastFingers ha supuesto un desafío integral que abarca la totalidad del ciclo de vida del desarrollo de software (SDLC). Lo que en su fase de ideación comenzó como una herramienta orientada a la práctica de la mecanografía, ha evolucionado a lo largo de su desarrollo hasta convertirse en un Producto Mínimo Viable (MVP) complejo, que integra dinámicas de gamificación competitiva y una arquitectura distribuida de nivel empresarial.

Este Proyecto Intermodular no solo consolida y valida las competencias troncales adquiridas durante el Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web (DAW), sino que ha exigido la investigación y aplicación proactiva de tecnologías emergentes que actualmente lideran la demanda en el sector tecnológico.

- **Balance Global y Cumplimiento de Objetivos**

El principal éxito del proyecto reside en haber logrado una convergencia perfecta entre el rendimiento técnico y la retención del usuario. Históricamente, el software de entrenamiento mecanográfico adolece de altas tasas de abandono debido a su enfoque puramente académico y monótono. FastFingers ha logrado revertir este paradigma inyectando lógicas propias de la industria de los videojuegos.

La implementación de un sistema económico cerrado (créditos), un catálogo de ventajas tácticas (Power-Ups), un "Salón de la Fama" global para fomentar el prestigio entre usuarios y la personalización mediante cosméticos y avatares, ha transformado el acto de teclear en una experiencia inmersiva y altamente competitiva.

- **Hitos Técnicos y Complejidad Arquitectónica**

A nivel de ingeniería, la plataforma se ha alejado deliberadamente de las soluciones monolíticas tradicionales, apostando por un diseño orientado a la escalabilidad y la alta disponibilidad. Los hitos más destacables son:

- **Arquitectura de Microservicios y API Gateway:** La decisión de segmentar el backend en microservicios (Usuarios, Incidencias, Lógica de Juego) orquestados mediante un servidor de descubrimiento (Eureka), demuestra una madurez técnica inusual en proyectos académicos. La implantación de un API Gateway ha sido fundamental no solo para el balanceo de cargas, sino como escudo de seguridad perimetral, gestionando de forma centralizada las complejas políticas de intercambio de recursos de origen cruzado (CORS) y blindando la topología de red interna frente al cliente.

- **Comunicaciones Full-Duplex en Tiempo Real:** Para soportar el "Modo Versus", se descartó el protocolo HTTP tradicional, el cual resulta ineficiente para el *polling* constante de datos. La adopción del protocolo STOMP sobre WebSockets ha permitido establecer un túnel de comunicación bidireccional y persistente. El servidor Spring Boot, actuando como Message Broker, sincroniza eventos críticos (como la cuenta atrás de los semáforos y el avance cruzado de las barras de progreso) con una latencia de milisegundos, gestionando además la liberación de recursos en red para evitar fugas de memoria (Memory Leaks) ante desconexiones abruptas, pero protegiendo el estado de la partida para el jugador activo.
- **Integración Resiliente de Inteligencia Artificial:** El modo "Partida a Medida" ha supuesto la integración exitosa de la API generativa de Google Gemini. El verdadero reto no fue el consumo de la API, sino la "Ingeniería de Prompts" (Prompt Engineering) para obligar al LLM a devolver formatos parseables, y la construcción de un sistema de *Fallback* o tolerancia a fallos. Si la IA externa experimenta caídas, el sistema inyecta datos locales de forma silenciosa, garantizando que el usuario nunca perciba una interrupción del servicio.
- **Frontend Modular y Orientado al DOM:** La elección de Vanilla JavaScript, prescindiendo intencionadamente de frameworks como React o Angular, persigue evidenciar un dominio profundo de los fundamentos de la web. La gestión manual del estado de la Single Page Application (SPA), la inyección y destrucción de nodos HTML, y el manejo de flujos asíncronos (*async/await*) para las llamadas a la API, han resultado en un cliente web extremadamente ligero, rápido y exento de dependencias de terceros.

- **Desafíos Afrontados y Crecimiento Profesional**

El desarrollo de FastFingers ha requerido horas de depuración exhaustiva y toma de decisiones críticas. Uno de los mayores desafíos fue el control de concurrencia y la monitorización de errores en un entorno distribuido. Esto se resolvió brillantemente aplicando el paradigma de Programación Orientada a Aspectos (AOP), que permite auditar excepciones y registrarlas en la base de datos de manera transversal, sin ensuciar la lógica de negocio ni interrumpir la experiencia del jugador.

- **Evolución del Proyecto: De la Propuesta Inicial al Producto Final**

Resulta fundamental analizar la trayectoria del desarrollo desde la propuesta entregada en noviembre hasta la implementación final. Un proyecto de ingeniería de software es un ente vivo que debe adaptarse a las pruebas de usabilidad y a la viabilidad técnica. En este sentido, se han tomado decisiones estratégicas para pivotar ciertas funcionalidades iniciales hacia un modelo más sólido y profesional:

- **Sustitución del sistema de "Vidas" por métricas de rendimiento:** En la propuesta inicial se contempló la implementación de vidas. Sin embargo, tras las primeras fases de pruebas, se determinó que un sistema de vidas penalizaba excesivamente el aprendizaje, frustrando al usuario. Se optó por un enfoque de "Sim-Racing" mecanográfico, donde el error no te expulsa de la partida, sino que penaliza tu precisión y velocidad (PPM), obligando al jugador a mejorar su técnica en lugar de simplemente intentar "sobrevivir".
- **Evolución de Habilidades a Power-Ups en "El Garaje":** La idea original de una selección de habilidades por tiempo se simplificó y profesionalizó integrándola en la economía del juego. En lugar de interrumpir la partida para elegir habilidades, se creó un sistema de inventario previo. Esto

permite que el jugador gestione su estrategia en el garaje, comprando ventajas pasivas (como el Escudo de Error o el Tanque de Nitrógeno), lo que genera un ciclo económico más claro y una jugabilidad más fluida.

- **Reducción de la duración de las partidas a 1 minuto:** Aunque inicialmente se plantearon sesiones más largas, la analítica de juego demostró que el pico de intensidad y concentración en mecanografía se alcanza en periodos cortos. Una duración de 60 segundos es el estándar de oro en la industria para medir las PPM reales, evita la fatiga muscular y mental del usuario y permite un emparejamiento (*matchmaking*) mucho más ágil en el modo online.
- **Cambio de "Battle Royale" a "Modo Versus" (1v1):** La propuesta mencionaba mecánicas de *Battle Royale*. Durante el desarrollo, se priorizó la calidad de la conexión y la latencia sobre la cantidad de jugadores simultáneos. Un sistema 1v1 mediante WebSockets garantiza una sincronización milimétrica y una experiencia competitiva pura. Implementar un Battle Royale habría diluido la competitividad directa y habría supuesto un riesgo para la estabilidad de la comunicación en tiempo real en esta fase del proyecto.
- **Sustitución de Logros por el "Hall of Fame":** Para fomentar una competitividad orgánica y sana, se decidió que el ranking global (Salón de la Fama) era una herramienta de retención mucho más potente que los logros estáticos. Ver el nombre propio en el podio frente a otros jugadores reales genera un incentivo de superación constante que un sistema de insignias tradicional no lograba alcanzar en las pruebas iniciales.

- **Escalabilidad y Futuras Líneas de Desarrollo**

La arquitectura actual sienta unas bases sólidas que permiten visualizar a FastFingers no solo como un proyecto académico, sino como un producto escalable. Las líneas de trabajo planteadas para una futura "Fase 2" incluyen:

- **Chatbot Inteligente con IA Contextual:** En la memoria se menciona un asistente virtual para resolver dudas frecuentes. Para la Fase 2, se propone evolucionarlo hacia un verdadero AI Coach o "Ingeniero de Pista", aprovechando la integración existente con la API de Google Gemini. En lugar de ofrecer respuestas predefinidas, el backend enviaría a la Inteligencia Artificial no solo la consulta, sino también las estadísticas detalladas del jugador (PPM, caracteres con mayor índice de error, ratio de victorias). El sistema podría sugerir áreas de mejora específicas y, mediante una implementación de RAG (Retrieval-Augmented Generation), generar automáticamente una partida a medida centrada en reforzar los puntos débiles detectados en la telemetría.
- **Telemetría Avanzada (Mapa de Calor del Teclado):** Ya se guarda la precisión y los créditos, pero se podrían exprimir más los datos para enriquecer el Perfil del Piloto. Registrando qué teclas específicas provocan más errores, en el panel del usuario de la Fase 2 se renderizaría un teclado virtual coloreado como un mapa de calor: las teclas en verde representarían los puntos fuertes, y las rojas indicarían las zonas donde se cometen la mayoría de los fallos. Esto aportaría una capa de visualización de datos (Data Visualization) de gran valor analítico.
- **Sistema Anti-Cheat (Protección de Integridad):** Cualquier entorno competitivo en navegador debe enfrentarse a la proliferación de scripts y bots de auto-teclado. Se implementaría un middleware en el frontend que analice la varianza del tiempo entre pulsaciones (keydown). Mientras que un humano mantiene un ritmo irregular y orgánico, un script pulsa

teclas con una precisión robótica o inyecta textos enteros de golpe mediante el portapapeles. El sistema detectaría estas anomalías, bloquearía el pegado de texto y anularía las partidas sospechosas, garantizando así la equidad del entorno competitivo y fortaleciendo la ciberseguridad general de la plataforma.

- **Sistema de "Coche Fantasma" (Ghost Racing):** Dada la fuerte inspiración en el mundo del motor, se plantea la introducción de mecánicas asíncronas de competición. En el modo Contrarreloj, al establecer un nuevo récord personal, el sistema guardaría un array JSON en la base de datos con el timestamp exacto de cada tecla pulsada. Al repetir esa prueba o intentar batir el récord de otro usuario del Salón de la Fama, la interfaz renderizaría una "barra de progreso fantasma" semitransparente que avanzaría exactamente al mismo ritmo que la carrera guardada, emulando la tensión del modo Versus en partidas individuales.
- **Escuderías (Sistema de Gremios/Clanes):** Para fomentar la retención a largo plazo y el sentido de comunidad, resulta necesario expandir el componente social del proyecto. Se permitiría la fundación de "Escuderías" (Clanes), donde los integrantes podrían aunar fondos para adquirir ventajas de grupo o participar en eventos periódicos, como "Carreras de Relevos". En estos eventos, las palabras correctas de todos los miembros sumarían a una barra de progreso global (gestionada mediante concurrencia por WebSockets) para obtener recompensas exclusivas.

- **Matchmaking Avanzado y ELO:** Evolucionar el actual sistema de salas privadas por código hacia un sistema automático de emparejamiento, donde algoritmos evalúen las Palabras por Minuto (PPM) históricas de los jugadores para enfrentar a usuarios del mismo nivel.
 - **Internacionalización (i18n):** Adaptar la plataforma a múltiples idiomas mediante diccionarios JSON dinámicos, abriendo la herramienta a mercados angloparlantes e internacionales.
 - **Ampliación de la Gamificación:** Implementación de "Modos Torneo", clanes y nuevos *Power-Ups* agresivos que afecten directamente la pantalla del rival (como oscurecer la interfaz o invertir los controles temporalmente), aumentando la capa estratégica.
-
- **Reflexión Final**

En conclusión, el desarrollo de FastFingers ha representado un reto técnico inmensamente enriquecedor que ha trascendido con creces el mero cumplimiento de los requisitos académicos estipulados. Lo que comenzó como un concepto para gamificar la práctica de la mecanografía, se ha materializado en un Producto Mínimo Viable (MVP) robusto, escalable y preparado para enfrentarse a las exigencias y estándares de un entorno de producción real.

El verdadero valor de este trabajo ha residido en la capacidad de orquestar y ensamblar piezas tecnológicas dispares bajo una misma arquitectura estructurada. La implementación de bases de datos relacionales seguras, el despliegue de un ecosistema de microservicios en Java (Spring Boot y Spring Cloud), la integración fluida de modelos de Inteligencia Artificial generativa y la construcción de un cliente web reactivo capaz de mantener comunicaciones Full-Duplex en tiempo real, han convergido en un producto final cohesivo, seguro y altamente interactivo. Afrontar y superar los problemas inherentes a los sistemas distribuidos ha supuesto un salto cualitativo en mi madurez técnica.

Desde la perspectiva del producto, FastFingers cumple estrictamente y con éxito sus objetivos pedagógicos y lúdicos. Se ha logrado transformar una actividad tradicionalmente monótona en una experiencia competitiva, demostrando que el diseño de interfaces (UI), la experiencia de usuario (UX) y las mecánicas de retención son tan críticas para el éxito de una plataforma como la solidez de su código subyacente.

En definitiva, este Proyecto Intermodular ha consolidado de manera definitiva mi perfil como Desarrollador Full-Stack. Esta memoria no solo documenta líneas de código o configuraciones de red, sino la capacidad real de abarcar el ciclo de vida completo del desarrollo de software: desde la toma inicial de requisitos y el diseño del modelado de datos, hasta la implementación de seguridad perimetral y la tolerancia a fallos. FastFingers se erige así no solo como el brillante colofón de mi etapa formativa, sino como una sólida e inmejorable carta de presentación, demostrando mi autonomía, capacidad de resolución de problemas y absoluta preparación para aportar valor de forma inmediata en el competitivo ecosistema laboral del sector tecnológico actual.

I.E.S. "ÁGORA"
C/ Lima, s/n 10005 CÁCERES
Teléfono: 927 00 67 16
Fax: 927 00 67 18
E-mail: ies.agora@edu.juntaex.es



**Modelo de Visto Bueno en la entrega de Módulo Proyecto Intermodular
de Ciclos Form. de GR. Superior DAW**

Nombre y apellidos del Profesor	Rubén Marín Silva
Nombre y apellidos del alumno/os	Jorge Barriga Rubio
Título del Proyecto	FastFingers

Grupo **2DAW**

A través del presente documento el Profesor titular del módulo acredita que el alumno citado anteriormente ha entregado en modo, forma y fecha el Proyecto Intermodular de Ciclos Formativos de Grado Superior DAW, arriba mencionado.

En Cáceres a 12 de Mayo de 2026

Vº Bº EL TITULAR

Firmado por MARIN SILVA RUBEN - ***2933** el día 12/05/2026 con un certificado emitido por AC FNMT Usuarios

Fdo.: Rubén Marín Silva